

3.9 - Circuit breaker e bulkhead

Impedir que a falha alheia vire a sua

Circuit breaker e seus estados

Falhar rápido em vez de esperar o timeout

- **Monitora a taxa de falha das chamadas a uma dependência**
 - Acima de um limiar, abre o circuito e para de chamar
- **Estado fechado deixa as chamadas passarem normalmente**
- **Estado aberto retorna fallback imediato, sem esperar**
 - Protege o pool de threads de ficar preso em timeouts
- **Meio-aberto testa a recuperação com poucas chamadas**
 - Volta a fechar somente se a dependência respondeu bem

Dois padrões, dois tipos de proteção

Circuit breaker

- Protege contra dependência lenta ou falha
- Falha rápido após cruzar o limiar
- Retorna resposta de fallback imediata
- Calibrado com base nos SLOs do serviço

Bulkhead

- Isola pools de recursos por tipo de chamada
- Falha em um pool não esgota os outros
- Inspirado nas anteparas de um navio
- Limita o raio de explosão de um problema

Calibrando um circuit breaker

JAVASCRIPT

```
const breaker = new CircuitBreaker(chamarRotasExternas, {  
  timeout: 2000,           // falha se passar de 2s  
  errorThresholdPercentage: 50, // abre com 50% de erro  
  resetTimeout: 10000      // testa meio-aberto apos 10s  
});  
  
breaker.fallback(() => rotaEstimadaLocal());  
  
breaker.on('open', () => log.warn('circuito aberto: rotas externas'));
```

Trinta segundos que derrubaram o checkout

- **API do Google Maps usada no cálculo de rotas**
 - Degradação faz a API responder com trinta segundos de timeout

LUNALOG

Numa tarde comum, a API do Google Maps que a LunaLog usava para calcular rotas começou a degradar, respondendo só depois de trinta segundos. Sem circuit breaker, cada cotação ficava parada esperando esses trinta segundos antes de falhar. Em noventa segundos, o pool de threads do serviço de rotas estava esgotado. Em dois minutos, o serviço de cotação saiu do ar, e levou junto o checkout de três e-commerces. Uma única dependência externa sem proteção foi suficiente para transformar a lentidão de um terceiro num apagão da plataforma inteira.



Serviços agora protegidos contra falhas das dependências. Mas o gargalo mais comum de todos, o que aparece mais cedo do que qualquer arquiteto gostaria, ainda nem foi abordado: o banco de dados. Toda a escala que construímos até aqui esbarra nele em algum momento. A pergunta é como fazer o banco crescer junto.

Estratégias de banco para larga escala